PA194HS: a hybrid PA-consistency machine

A comparison of the supplied designs, a distinct 194-state construction, and reproducible mathematical and physical certificates

Research prepared for Vladimir Reshetnikov

13 November 2026, Pacific time

Abstract

No machine with fewer than 194 working states is established here. The result is the requested alternative: **PA194HS**, a different 194-state, two-symbol, single-tape deterministic machine, starting on a blank two-way tape. Its ordinary mathematical calibration gives

$$\mathrm{PA} \vdash \big(\neg\,\mathrm{Halt}(\mathrm{PA194HS}, \mathrm{blank}) \leftrightarrow \mathrm{Con}(\mathrm{PA})\big).$$

It combines no-push, top-result modus ponens with a cache-only contradiction test, and uses symmetric destructive comparison in the physical realization. The new command combination has a one-command discard and an explicit proof-word compiler valid over arbitrary incoming stores and numerical Cantor stacks, including their zero aliases. A second table, PA194H, realizes the same command interface without the symmetric optimization.

The comparison is pinned to ProveIt commit `1e407ae841e4666f932777fad977ba784ebbcfd5`. It covers 46 research packets and recognizes 666 literal table byte forms, including 51 forms in 28 existing 194-state graph classes. Both new tables are absent even up to state renaming, choice of root, tape reflection and global symbol exchange. PA194HS has 3,650 primitive source sites and 4,750 universally checked outcomes. Its exact whole-tape blank initialization takes 57,768 transitions.

The result uses the explicitly identified, inherited syntactic PA calibration of the separate-head numerical calculus. The new command proof, literal tables, two physical checking implementations, source maps, complete comparison data and bounded unsuccessful reduction searches are supplied. No new end-to-end Lean or Rocq theorem, completed iPRA endpoint, or global lower bound is claimed.

Contents

1 Result and exact machine convention **3**

2 Comparison of the supplied designs **4**

2.1 Scope and identity 4

2.2 Main families and compatible savings 4

2.3 The new command interface is not an old table under new names 5

3 The numerical calculus retained by the construction **6**

4 A hybrid controller with a new proof interface **7**

4.1 Numerical commands 7

4.2 Uniform word identities 8

4.3 The new MP suffix and proof compiler 9

4.4 Soundness and PA-internal calibration 10

4.5 Exact source binding and the symmetric implementation 10

5 Why the inherited calibration is a PA-internal argument **11**

5.1 From numerical proofs to PA proofs 11

5.2 From PA refutations to numerical proofs 12

6 Physical realization and exact verification **12**

6.1 The native patch and its allocation 12

6.2 Universal contracts, not bounded register samples 12

6.3 Initialization is an independent obligation 13

7 Attempts to reduce the count below 194 **13**

7.1 Lessons from ZFC and ConciseIPRA 14

8 Reproduction, package contents and limitations **14**

A The complete PA194HS transition table **16**

B All 46 research packets at the pinned snapshot **17**

C All existing 194-state action-graph classes **19**

1 Result and exact machine convention

The primary deliverable is `machines/PA194HS/machine.tm`. Its working states are $0, \ldots, 193$, its initial state is 0, and its separate terminal halt label is 194. The head begins at cell zero of the all-zero tape indexed by $\mathbb{Z}$. Every transition reads and writes one bit and moves exactly one cell left or right. The table has 388 rows and no outgoing rows for halt. State counts exclude halt. The complete table is printed in Appendix A; the authoritative five-column format is

```
state  read  write  L-or-R  next-state.
```

Its SHA-256 is

`19935635f2c5722f5a72977d20a7019764a28573e12d1d0bce398245576c33b1`.

The companion `machines/PA194H/machine.tm` has the same conventions and SHA-256

`3276a34fd0388102dad87a8b12c70e88b2bcccf6b068cb034ce78d33536adcc1`.

Native compiler labels are supplied separately, with initial state 74 and halt 254. Exact bijections between native and dense tables are checked.

Theorem 1.1 (Ordinary mathematical PA calibration)**.** *For each of the two literal tables M delivered above,*

$$\mathrm{PA} \vdash \mathrm{Halt}(M, \mathrm{blank}) \leftrightarrow \exists p \, \mathrm{Prf}_{\mathrm{PA}}(p, \ulcorner 0 = 1 \urcorner),$$

and therefore $\mathrm{PA} \vdash \neg \mathrm{Halt}(M, \mathrm{blank}) \leftrightarrow \mathrm{Con}(\mathrm{PA})$.

Here Halt is the existential arithmetic assertion of a finite literal transition trace, and $\mathrm{Prf}_{\mathrm{PA}}$ is the usual primitive recursive proof predicate for a fixed standard presentation of PA. Changing to another standard presentation uses the ordinary effective proof translations. The proof below composes an inherited numerical-calculus calibration with a new command compiler and freshly checked physical realizations. It is an ordinary proof of provability in PA; its displayed statement is not presented as the output of a newly compiled proof assistant.

Quantity	PA194H	PA194HS
Working states / transition rows	194 / 388	194 / 388
Controls before folding	201	202
AX readers	67	67
Simultaneous folds	7	8
Primitive source sites	3,653	3,650
Source outcomes, each checker	4,754	4,750
Independent symbolic steps	1,097,251	1,101,153
Exact blank initialization	57,922	57,768

The HS count is $135 + 67 - 8 = 194$: the symmetric controller adds one reader/control, then its compatible fold removes one. H has $134 + 67 - 7 = 194$. Every working control is reachable in the transition graph; this is not a claim that each is visited by the unique blank computation. The shorter startup is only a comparison of these finite initialization segments, not a uniform running-time claim.

2 Comparison of the supplied designs

2.1 Scope and identity

The pinned PA directory contains 41 filed research packets; the current incoming directory contains five more archives. The 46-packet ledger is printed in Appendix B. The numerous `pa194-*` proof directories outside `docs/` are components of formalization work for the same primary table, rather than additional machine designs.

The fresh recognizer accepts complete numeric binary tables in `.tm`, `.txt`, `.tsv`, and `.csv`, both in the PA tree and recursively inside incoming and nested ZIPs. It requires one transition per working-state/read-bit pair and exactly one undefined terminal target. Tables encoded only in Lean, Python, JSON, PDFs or prose are outside this literal-table count; packet descriptions and source arguments are compared separately. Byte-identical nested ZIP payloads are traversed once.

Observed quantity	Count
Top-level research packets	46
Recognized literal table occurrences	3,741
Distinct table byte forms	666
Root-spanning action-graph classes	476
ZIP references / distinct ZIP payloads	31 / 19
194-state occurrences / byte forms / graph classes	261 / 51 / 28
194-state principal or described companion graph classes	16
194-state supplementary or research graph classes	9
194-state explicitly rejected graph classes	3

Two disconnected synthetic controls, with six and nine states, are excluded from the graph-class count. There is no recognized non-synthetic table of fewer than 194 working states. These are reproducible observations about a specified corpus, not a lower bound or 476 consistency theorems.

Each action graph retains read-bit labels, write bits, movements and the halt target. Canonicalization tries every root reaching the full working graph and numbers vertices in ordered breadth-first discovery. Novelty is checked by equality of the full canonical tuples, with reflection and simultaneous read/write symbol exchange included. Both new tables have no match against all 51 existing 194-state byte forms. This graph test rules out cosmetic renaming; the command contracts below establish the additional semantic distinction.

2.2 Main families and compatible savings

Family	States	Distinction and comparison finding
U208 families	208	Finite-history, entry and three-cell presentations repeatedly reuse the same contextual fold. Named outputs cannot be counted as independent state savings.
Imported PA209/PA196	209/196	James Chang's numerical proof search and physical compiler are the principal upstream foundation. The imported internal PA implication from nonhalting to consistency does not by itself give its converse.

Family	States	Distinction and comparison finding
Original PA195	195	Bundled diagonal-vacuity emitter; 3,700 source sites. The later separate-head source changes both the emitter and stack interface.
PA195S / PA195C	195	Separate-head source, with original or cached controller; 3,673 or 3,649 sites. Raw allocation and compatible folds matter together.
M195S8	195	An additional scanner state repairs a proposed 194-state quotient whose source addresses contain a forbidden zero run.
Primary PA194	194	Same separate-head source as 195S; $134 + 67 - 7$. The A/D register permutation and a $134 + 68 - 8$ decoder are distinct companions.
PA194C / PA194CS	194	Cached-head contradiction test and cached-result MP; CS uses symmetric destructive equality. Five reports repeat the same principal C table.
PA194N	194	No-push MP returns the conclusion in the visible top and offset, retaining the original sum-based Shift test.
PA194Q / PA194D	194	Q exchanges the premises of quantifier distribution. D moves the separate diagonal head to another parameter and uses 69 readers with nine folds.
PA194J / F / A / X	194	Proved-zero decrements replace selected unconditional source jumps. X is literally the same table as the Q packet's PA194CSZ.
PA194R	194	Final known-zero decrement plus a different reader association; source-address and subtree differences are substantial, but the count remains 194.
M4R / M5 / M6	194	Reassociated coefficient-4 design and formula-constructor coefficients 5 and 6. Coding changes require explicit logical proof transport.
PA194H / PA194HS	194	New combination: no-push, top-result MP with cache-only Shift. HS implements that same interface using symmetric comparison.

The full 28-class frontier, including rejected and supplementary tables, is in `comparison/frontier194.csv`. Two exact duplicates deserve attention: repeated PA194C presentations share dense hash beginning `d759313c03b5`; PA194CSZ and the latest PA194X share `62e93098caf7`. Our comparison includes the five newest incoming packets, whereas their own older novelty audits used an earlier snapshot.

2.3 The new command interface is not an old table under new names

Let $b = d + x$ and assume successful MP, $I(t, b) = h$; numerical stack notation is defined in the next section. The three existing protocols and the new one compare as follows.

Protocol	Successful MP output	Shift halts when
Primary PA194	$(b, [0; w], b, 0)$	$t + h = 1$
PA194N	$(b, w, b, 0)$	$t + h = 1$
PA194C / CS	$(0, w, 0, b)$	$h = 1$
New H / HS	$(b, w, b, 0)$	$h = 1$

The new controller composes supplied ideas rather than claiming either primitive idea as novel. Its

contribution is an independently checked 194-state realization and a correct proof compiler for the combined interface. In particular the cached-result proof compiler cannot simply be copied: the new successful MP leaves $d = b$, and therefore needs a different cleanup word. Conversely, the no-push source's old Shift test is no longer the halt predicate. The new Discard clears an arbitrary cache in one command. This is a proof-interface simplification and a different machine design; it is not an improvement over 194 states.

3 The numerical calculus retained by the construction

The construction retains the exact separate-diagonal-head axiom emitter, its parameter guards, six other logical heads and dominated propositional head. This section specifies that dependency so that a changed machine controller is not mistaken for a new unproved consistency theory.

For Cantor pairing $p(a, b) = (a + b)(a + b + 1)/2 + b$, let

$$I(A, B) = 4p(A, B), \quad M(a, b) = 4p(a, b) + 1, \quad U(a, B) = 4p(B, a) + 2, \quad E(a, b) = 4p(a, b) + 3.$$

Code zero has the exceptional reading truth; every positive code is decoded by its residue and the Cantor projections. $M(a, b)$ and $E(a, b)$ are atoms with variable labels a, b, and $U(a, B)$ binds label a in the decoded formula B. The target contradiction is code $1 = M(0, 0)$. Set $N(B) = I(B, 1)$, use classical implicational conjunction, and define $\exists x_a B = N(U(a, N(B)))$. The alias $I(0, 0) = 0$ is retained explicitly.

The proof calculus has truth 0, modus ponens from A and $I(A, B)$ to B, generalization from B to $U(a, B)$, the dominated implicational schema

$$\rho_8(D, A, H, d) = I(I(I(H, d), D), I(I(D, H), I(A, H))), \qquad d \le D,$$

the diagonal schema $\Gamma(B) = I(B, U(B, B))$, the following six schemas, and the guarded gate described next. Implications in this display denote the numerical constructor I. Truth 0 already follows from $\rho_8(0, 0, 0, 0) = 0$, so listing it separately adds no strength.

$$\begin{aligned}
L_1(a, C) &= (\exists x_a\, U(a, C)) \to C,\\
L_2(a, b, C) &= U(a, U(b, C)) \to U(b, U(a, C)),\\
L_3(a, b, c) &= M(c, a) \to (E(c, b) \to M(b, a)),\\
L_4(a, b, c) &= E(c, a) \to (M(b, c) \to M(b, a)),\\
L_5(a, c) &= \exists x_a E(c, a),\\
L_6(a, b, c) &= U(a, b) \to \big(U(a, I(b, c)) \to U(a, c)\big).
\end{aligned}$$

Define

$$\begin{aligned}
V(a, b, c) &= I(M(a, b), U(c, M(a, b))), & P(a, b, c) &= I(c, M(c, b)),\\
S(a, b, c) &= U\big(b, I(M(a, b), P(a, b, c)) \wedge I(M(c, b), c)\big),\\
Q(a, b, c) &= U\big(c, I(S(a, b, c), N(P(a, b, c)))\big),\\
G(a, b, c) &= N(I(V(a, b, c), Q(a, b, c))).
\end{aligned}$$

The gate is $K(a, b, c) = G(a, b, c)$ when $c \neq a, b$, and zero otherwise. The symbol c serves both as a number coding a formula and as a variable label; that intentional coincidence is the source of the short diagonal schema. Every variable label occurring in a formula of code B is strictly below B. Hence x_B

is absent from that formula and $\Gamma(B)$ is a valid vacuity instance. The zero case is handled separately, not by a false strict inequality for every constructor.

The free-term emitter check in `construction/mathematics/check_hybrid_words.py` checks the actual 190-token emitted word and ordered stack, including the old cached generalization and the dominated parameter. It uses the source's exact parameter conventions. The rule inventory above specifies the logical families; it does not replace that binding by an unordered set of formulas.

4 A hybrid controller with a new proof interface

The new machines combine two previously supplied operations whose combination is not one of the supplied command interfaces. Modus ponens returns its conclusion in the visible top, preserves the offset containing that conclusion, and pushes no dummy cell. Shift tests only the cached head. PA194H realizes these commands directly; PA194HS uses the supplied symmetric comparison optimization to implement exactly the same commands with three fewer primitive sites. The new contribution is the combination, its exact proof compiler and its literal 194-state implementations. Neither the no-push idea nor the symmetric comparison is claimed as new in isolation.

4.1 Numerical commands

Write $p(a,b) = (a+b)(a+b+1)/2 + b$, $[a;w] = p(a,w)$, $I(a,b) = 4p(a,b)$ and $U(a,B) = 4p(B,a) + 2$. The total projections satisfy $\mathrm{hd}[a;w] = a$, $\mathrm{tl}[a;w] = w$, and $\mathrm{hd}(0) = \mathrm{tl}(0) = 0$. In particular $[0;0] = 0$. No stack-length argument below assumes otherwise.

A command store is (t,w,d,h). Its two changed commands are

$$S(t,w,d,h) = \begin{cases} \mathsf{halt}, & h = 1, \\ (h, \mathrm{tl}\, w, d, \mathrm{hd}\, w), & h \neq 1, \end{cases}$$

and, writing $b = d + x$,

$$\mathsf{MP}_x(t,w,d,h) = \begin{cases} (b,w,b,0), & I(t,b) = h, \\ (0,w,b,0), & I(t,b) \neq h. \end{cases}$$

AX is unchanged from the separate-diagonal-head calculus. It pushes ten entries, including the saved old top. Put $D = d + x$, $H = h + y$, and $A = c$. In stack-top order its nine generated entries are

$$(q_1, \ldots, q_9) = (\Gamma(A), L_6, L_5, L_4, L_3, L_2, L_1, \rho_8(D,A,H,d), U(d,h)).$$

Here $\Gamma(B) = I(B, U(B,B))$, the six L_i are the unchanged universal heads, ρ_8 is the unchanged dominated implicational head, and the visible output is the unchanged guarded separation gate $K(H,D,A)$. Thus

$$\mathsf{AX}_{c,x,y}(t,w,d,h) = (K(H,D,A), [q_1, \ldots, q_9, t; w], 0, 0).$$

All $q_1, \ldots, q_8$ and the gate have residue zero modulo four; q_9 has residue two. Hence none equals one, including the failed-guard case where $K = 0$. The executable free-term check verifies the entire emitter stack and top, not merely this residue observation.

4.2 Uniform word identities

Words execute from left to right. For every incoming store,

$$\mathsf{AX}_{c,x,y}; S^9(t,w,d,h) = (\rho_8(D,c,H,d),[t;w],0,U(d,h)),$$
$$\mathsf{AX}_{c,x,y}; S^{10}(t,w,d,h) = (U(d,h),w,0,t).$$

These executions never halt: they test only the cleared initial cache and the nine generated heads, not the saved arbitrary t.

Define

$$\mathsf{Push} = \mathsf{AX}_{0,0,0}; S^9; \mathsf{MP}_0, \quad \mathsf{Pop} = \mathsf{AX}_{0,0,0}; S^{11}, \quad \mathsf{Discard} = \mathsf{MP}_0.$$

The exact contracts are

$$\mathsf{Push}(t,w,d,h) = (0,[t;w],0,0),$$
$$\mathsf{Pop}(t,[a;w],d,h) = \begin{cases} \mathsf{halt}, & t = 1, \\ (t,w,0,a), & t \neq 1, \end{cases}$$
$$\mathsf{Discard}(t,w,0,h) = (0,w,0,0).$$

For Push, the final MP compares a residue-zero implication with the residue-two universal head and therefore fails. Pop's eleventh Shift tests precisely the saved top, and is its only possible halt. For Discard, the candidate conclusion is zero; both the success and failure branches return the same store. Consequently Discard clears an arbitrary old cache without any equality side condition. This is shorter than the multi-command discard required by the supplied cached-result controller.

The empty-stack alias is harmless: every displayed equality is an equality of the total numerical projections. In particular Push sends an all-zero store to an all-zero store; it does not create a distinguishable zero cell.

Let $\mathsf{Offset}_i = \mathsf{MP}_i$. From $(0,w,0,0)$ it returns $(0,w,i,0)$ for every i, including zero. Generalization is compiled as

$$\mathsf{Gen}_i = \mathsf{Push}; \mathsf{Offset}_i; S; \mathsf{AX}_{i,0,0}; S^{10}.$$

It sends $(a,w,0,0)$ to $(U(i,a),w,0,0)$, even for $a = 1$. The sole Shift before AX tests zero, then places a in the cache; AX consumes that cache before any further Shift.

For $1 \leq j \leq 7$, the ordinary head selector

$$\mathsf{Discard}; \mathsf{AX}_{A,D,H}; S^{j+1}; \mathsf{Pop}^{9-j}$$

sends arbitrary $(t,w,0,0)$ to $(q_j,w,0,0)$. The selected head has residue zero and so every Pop is safe. The last consumed stack entry is the saved zero. The diagonal head is $j = 1$, giving explicitly $\mathsf{Discard}; \mathsf{AX}_{B,0,0}; S^2; \mathsf{Pop}^8$ for $\Gamma(B)$. The gate selector is

$$\mathsf{Discard}; \mathsf{AX}_{c,b,a}; \mathsf{Pop}^{10}.$$

For $u \leq a$, the dominated implicational selector is

$$\mathsf{Discard}; \mathsf{Offset}_u; \mathsf{AX}_{b,a-u,c}; S^9; \mathsf{Pop}.$$

It returns $\rho_8(a,b,c,u)$ with the original stack and cleared caches.

4.3 The new MP suffix and proof compiler

Define

$$\mathsf{MPSuffix}_b = S; S; \mathsf{MP}_b; \mathsf{Pop}.$$

For arbitrary a, b, w it obeys the exact alternative

$$\mathsf{MPSuffix}_b(0, [a, I(a,b), 0; w], 0, 0) = \begin{cases} \mathsf{halt}, & a = 1 \text{ or } b = 1, \\ (b, w, 0, 0), & a \neq 1 \text{ and } b \neq 1. \end{cases}$$

The first Shift loads a into the cache. The second either halts or establishes $(a, [0; w], 0, I(a,b))$. MP succeeds, leaving $(b, [0; w], b, 0)$. The final Pop either detects $b = 1$, or consumes the reserved zero and clears the offset without changing b or the old stack. Reserving this zero is necessary: without it, Pop would consume the first entry of the arbitrary underlying stack. At $a = b = w = 0$ the same calculation applies, because $I(0,0) = 0$ and $[0;0] = 0$.

Compile each finite numerical proof tree p into a word W_p. Use the selectors above for axioms and $W_{p_0}; \mathsf{Gen}_i$ for generalization. At an MP node with minor premise a, major premise $I(a,b)$, and subproofs p_0, p_1 respectively, set

$$W_p = \mathsf{Discard}; \mathsf{Push}; W_{p_1}; \mathsf{Push}; W_{p_0}; \mathsf{Push}; \mathsf{MPSuffix}_b.$$

Induction on the proof tree proves that for every t, w, execution from $(t, w, 0, 0)$ either halts early or returns $(\mathrm{concl}(p), w, 0, 0)$. The first two commands reserve the dummy zero. The next two compiled proofs and Pushes establish the exact suffix precondition. Generalization tolerates a premise equal to one, and every earlier halt remains a halt under word extension.

If p proves one, then

$$\mathsf{AX}_{0,0,0}; W_p; \mathsf{Pop}$$

halts from *every* incoming store. The preliminary AX clears d, h; the compilation invariant tolerates its arbitrary resulting top and stack. If no earlier halt occurs, the final Pop tests the proven top one before it depends on any value of the underlying stack. This dirty-store property is the precise interface needed by the unchanged exhaustive command-word scheduler, whose state is carried between trials.

For completeness, the scheduling argument is finite and cofinal, rather than an assertion that every trial starts with an empty store. The inherited command encoding has

$$B_S(z) = p(0, z), \qquad B_{\mathsf{MP}_x}(z) = p(1, p(x, z)), \qquad B_{\mathsf{AX}_{c,x,y}}(z) = p(c+2, p(y, p(x, z))).$$

For continuation words define $E(\epsilon) = 0$ and $E(c :: w) = 1 + B_c(E(w))$. Put $z = E(\mathsf{AX}_{0,0,0}; W_p; \mathsf{Pop})$. Prefixing that universal halting word by $\mathsf{AX}_{k,0,0}$ gives the fresh-candidate codes

$$n_k = p(k+2, p(0, p(0, z))).$$

They are strictly increasing and unbounded in k. Their prefix command can change the store arbitrarily, because the following word halts from every store. The fresh candidate counter starts at two, so omitting zero and one loses none of this cofinal family. For any positive continuation l, the scheduler decodes $l - 1$ and its next residual continuation is at most $l - 1$. Every finite trial therefore terminates or halts; the unchanged arithmetic source loops also have primitive-recursive terminating bounds. Bounded induction through the earlier candidate numbers reaches one chosen n_k, unless a halt occurs first. The continuation distinction is essential: $E(w)$ is a positive continuation code, while the first candidate is its outer command code without the added one. PA proves the coding identities and finite execution induction directly.

4.4 Soundness and PA-internal calibration

Maintain a numerical-calculus derivation for the visible top, cached head and every live entry in the canonical finite Cantor stack. Code zero has the fixed derivation $\rho_8(0,0,0,0) = 0$; it supplies the virtual zero tail. No derivability invariant is imposed on the offset d.

AX preserves this annotation by the unchanged axiom instances and theorem generalization. MP success obtains b by modus ponens from the annotated minor t and major $I(t,b) = h$; MP failure returns the fixed truth code. Shift merely moves annotated entries and halts only on an annotated proof of one in the cache. The annotation updates and the compiler above are primitive recursive operations on finite objects. Ordinary induction in PA verifies their correctness. No reflection or soundness axiom for PA is used.

The unchanged separate-head logical calibration supplies PA-verifiable proof translations between refutations of PA and proofs of numerical code one. Its original mathematical texts are retained in `references/logical/`; the supplied project's missing end-to-end Lean/iPRA reductions are not treated as completed. Combining those inherited translations with the new command proof and unchanged exhaustive scheduler proves, in PA,

$$\mathrm{Halt}(R_H) \iff \exists p \ \mathrm{Prf}_{\mathrm{PA}}(p, \ulcorner 0 = 1 \urcorner).$$

This is an ordinary mathematical proof of a PA-provability statement, not a newly compiled proof-assistant theorem.

4.5 Exact source binding and the symmetric implementation

The source has nine logical registers $(N, T, W, A, D, H, X, Y, L)$. The new MP subtree replaces `node163=(node6,node55)` with `(node6,node54)`. As in PA194N, it removes the dummy-zero push; on success the retained seven-site copy-and-restore tail sends $(T, D, H, X) = (0, b, 0, 0)$ to $(b, b, 0, 0)$ in $7b + 2$ primitive steps.

The Shift source in PA194H first clears the old top, copies H into T and X, restores X to H, increments X and jumps to the old root comparison. From arbitrary t, h with $X = 0$, it reaches $(T, H, X) = (h, h, 1)$. The unchanged destructive equality guard halts just when $h = 1$; otherwise it clears both compared registers and continues to the unchanged Cantor decoder. This is exactly the displayed cache-only Shift command.

PA194HS instead omits the restore loop and increments H, arriving at $(T, H, X) = (h, 1, h)$. Equality is symmetric, and on inequality the guard clears both operands. Therefore its surviving registers and subsequent source control are identical to PA194H. The change removes three source sites. Its additional physical fold is $41 \mapsto 330$, with actions $0L31$ and $1L46$; all four local history/read-bit cases are rechecked in the final machine together with the seven inherited AX folds.

The literal constructors produce PA194H with 3653 sites and 4754 source outcomes, and PA194HS with 3650 sites and 4750 outcomes. Their entire resolved source graphs and complete tables are delivered. Each outcome has a universal symbolic physical contract over arbitrary natural registers, both decrement branches, arbitrary affine header gap and an arbitrary nonempty finite spare list. The independent verifier derives the four register-list lemmas again from the literal rows. Exact blank initialization reaches the clean source site 25 with logical file $(2, 0, \ldots, 0)$ after 57922 and 57768 transitions respectively; the complete finite-support tape is compared, including every spare block.

Positive physical durations give both directions of finite-trace equivalence. A source halt is simulated by concatenating finite positive segments. Conversely, if the physical run first halted at time T while

the source had not halted, the first $T + 1$ source steps would produce a longer nonhalting physical trace, contradicting determinism. This finite argument and the loop certificates are formalizable in PA. Consequently, for either literal table M,

$$\mathrm{PA} \vdash (\neg\, \mathrm{Halt}(M, \mathrm{blank}) \longleftrightarrow \mathrm{Con}(\mathrm{PA})).$$

Why this is a different command design. The original no-push PA194N halts on Shift from $(1, 0, 0, 0)$, whereas the hybrid clears that store without halting. The original cached-result PA194C sends $(0, w, 0, 20)$ under MP_2 to $(0, w, 0, 2)$, whereas the hybrid sends it to $(2, w, 2, 0)$, since $I(0, 2) = 20$. The new proof compiler accounts for both differences. These changes affect the specified command behavior, rather than merely renaming states, tape directions or formula parameters.

5 Why the inherited calibration is a PA-internal argument

The new controller proof does not assume that semantic equiconsistency alone supplies a theorem of PA. The inherited calibration constructs finite syntactic translations in both directions. Its original texts, including the diagonal and separate-head preservation arguments, are retained under `references/logical/`, with their provenance. The following gives the mathematical substance and the exact reliance on those results.

5.1 From numerical proofs to PA proofs

Translate $M(a, b)$ as finite binary membership $\mathrm{Bit}(x_a, x_b)$, $E(a, b)$ as equality, and $U(a, B)$ as universal quantification of the translated B. A suitable formula is

$$\mathrm{Bit}(s, z) \equiv \exists u, v\, [s = (2u + 1)2^z + v \ \wedge \ v < 2^z].$$

Exponentiation may be treated by its standard definitional extension in PA. PA proves $\mathrm{Bit}(s, z) \to z < s$, and hence refutes $\mathrm{Bit}(x_0, x_0)$, the translation of code one.

For every formula $\theta(z, \vec{u})$, PA proves finite binary comprehension:

$$\forall n\, \exists s < 2^n\, \forall z < n\, (\mathrm{Bit}(s, z) \leftrightarrow \theta(z, \vec{u})).$$

Induct on n, retaining s when $\neg\theta(n)$ and adding 2^n when $\theta(n)$. Substituting the syntax of θ into this induction template is a primitive recursive operation producing a PA proof. This is a statement about generated proofs, not a uniform truth predicate.

This template interprets pointed separation in the gate. Distinct labels a, b use the bounded matrix $(\mathrm{Bit}(x_a, z) \vee z = x_b) \wedge T(c)[z/x_b]$ with a bound such as $x_a + x_b + 1$. When $a = b$, the binder captures both occurrences of that variable, so the correct matrix is $(z = x_b) \wedge T(c)[z/x_b]$; the other membership occurrence becomes $\mathrm{Bit}(z, z)$ and is refuted. The guard supplies freshness of label c. The diagonal head uses code freshness. The remaining heads have uniform logical proof templates. The collision $I(0, 0) = 0$ is handled by the fixed equivalence of truth with truth implying truth.

Composing these templates with the annotation argument of Section 4 turns a finite literal halt into a finite PA refutation. Thus the direction from a source halt to inconsistency uses no assertion that PA is sound or consistent.

5.2 From PA refutations to numerical proofs

The imported PA196 refutation compiler uses equality through reverse membership rewriting, neighborhood equality and an equality witness. The supplied diagonal-calculus argument recovers those same interfaces after replacing equality vacuity by $\Gamma(B)$.

For reverse rewriting take the negation φ of the desired implication $E(a,b) \to (\chi(b) \to \chi(a))$, with χ a membership atom in the needed slot, and choose the separator label to be the formula code $c = \ulcorner\varphi\urcorner$. The label is fresh by the code bound. Pointed separation, direct rewriting, the retained equality witness and membership-only vacuity derive $\varphi \to Q$. The diagonal instance $\Gamma(\varphi)$ supplies the required quantification over c. The same separation instance supplies $N(Q)$; classical reasoning gives the desired reverse implication. Coincident labels are treated separately.

For neighborhood equality, choose first the fresh binder $z_0 = \ulcorner E(a,b)\urcorner$. Its diagonal instance and two-way membership rewrites give neighborhood equivalence at z_0; the retained witness and membership vacuity transport the binder to the required other labels. These finite templates replace the equality interfaces in the existing PA-refutation compiler. The supplied separate-head proof then translates the formerly bundled diagonal gate to its independent diagonal clause and old gate, with the ten-entry stack convention used here. The operations on codes and finite derivations are primitive recursive and their correctness is proved by induction in PA.

This second logical translation is an inherited mathematical dependency; we do not claim to have newly formalized its full closure. The fresh contribution starts at the combined command interface and its physical realizations. The project README's open machine-specific iPRA reductions remain open: ordinary mathematics, executable symbolic certificates, and a kernel-checked internal theorem are different evidence levels.

6 Physical realization and exact verification

6.1 The native patch and its allocation

Both machines use the same 190-token AX word and 67-reader grammar as primary PA194. H uses five established cache-only Shift edits together with the no-push MP edit $\delta(164,1) = (1,R,114)$, replacing the old target 98. HS removes the Shift restoration loop, increments the other comparison operand, and adds the contextual fold $41 \mapsto 330$. The complete source-node edits and every changed native row are in the deterministic builder and receipts. The allocation is explicit; there is no interpreter, prewritten input tape or uncounted bootstrap machine.

The extra fold uses actions $0L31$ and $1L46$ at the surviving control. Its two possible entry-history bits and two read bits give four local cases. Together with the seven inherited folds this is 32 cases for HS and 28 for H, checked against the final simultaneously folded table. Checking each pair only against its own intermediate parent would not suffice, because folds can interact through targets and predecessor rows.

6.2 Universal contracts, not bounded register samples

A canonical source frame has a binary instruction address, affine zero gap, marker and unary register list. The physical order of the nine logical registers is the permutation

$$(6,2,0,4,3,7,8,5,1).$$

The contract domain quantifies over all nine natural register values, both outcomes of a decrement, every header distance $48 + n$ for $n \geq 0$, and an arbitrary nonempty finite list of natural spare values. The first checker uses supplied arithmetic-block contracts; the independent checker rederives the four generic register-list lemmas from the literal rows. Its two source resolvers also agree on every instruction and successor.

The independent engine takes 1,101,153 symbolic steps for HS and checks all 4,750 source outcomes. The largest one-outcome count is 403; the minimum positive constant in an outcome-duration expression is 3. The corresponding numbers for H are 1,097,251, 4,754, 403 and 3. The separate construction checker obtains the same source effects using different symbolic representations. These finite computations support the universal contracts through the stated loop inductions. Their symbolic rules and generic induction arguments are part of the mathematical verification base; Python success alone is not a proof assistant kernel theorem.

Positive durations are needed for the reverse simulation. Given a physical halt at time T, if the total source had not halted in its first $T + 1$ steps, composing those positive nonhalting segments would produce a deterministic physical trace longer than T, a contradiction. This is a finite argument available inside PA, without an appeal to an infinite source execution in the standard model.

6.3 Initialization is an independent obligation

Two separate concrete simulators check the entire finite-support tape. For HS, after exactly 57,768 transitions, native control 84 and head -2 reach source site 25, with logical register file $(2, 0, \ldots, 0)$, 26 one-valued spare blocks, the intended markers, source bits and zero exterior. H reaches its corresponding complete frame in 57,922 transitions. No earlier halt occurs. This establishes the bridge from the actual blank input, rather than from an assumed clean source frame.

This distinction excludes real failures in the supplied corpus: a source-equivalent known-zero-jump modification passes later source contracts yet halts after 1,361 bootstrap transitions. Neither the new tables nor their proofs rely on that rejected modification.

7 Attempts to reduce the count below 194

The search tried a one-state quotient on both primary PA194 and the new PA194HS. In each case 23 arithmetic-block controls were protected and 171 other controls were eligible. For each unordered unprotected pair, the surviving state's two rows were independently chosen from the corresponding rows of either original state. This gives $4\binom{171}{2} = 58,140$ candidates. A further $171 \cdot 23 = 3,933$ candidates identify an unprotected control with a protected one while retaining the protected rows. Incoming targets are redirected and other rows remain unchanged: **62,073 candidates per table, 124,146 total**.

Each candidate is screened by the complete effects of affected source contracts, with a 1,024-symbolic-operation limit per attempted contract. The original contract's visited controls determine which contracts can change. A contract touching neither altered control follows an unchanged run. Thus this screen permits larger detours than a one-transition local simulation, but restricts the candidate rows to the stated choices. Neither input produced an accepted quotient. Two additional local-action screens on primary PA194 used 16,393 radius-five windows with a 300-step bound, and 29,347 radius-six windows with a 1,000-step bound, and produced no candidate. These window sets are search inputs, not a separately proved complete reachable-context theorem.

The most instructive direct attempt deletes native control 34 and redirects control 23's zero row to

75. This shortens the return scanner from seven consecutive zeros to six and gives 193 controls. On HS, 113 of 4,750 symbolic source contracts, involving 57 sites, fail to close within the stated bound. Both branches at site 0 fail, along with the scheduler entry and many AX entries. Temporary operation markers turn some clean-address trailing zero runs into internal zero runs, so counting runs in clean source addresses is insufficient.

There is also an exact finite failure witness, independent of symbolic timeout. At source site 0, path `000000`, gap parameter zero, and ten zero-valued physical register/spare blocks, the original table's second return to native interpreter control 84 has head position zero (at transition 246). The shortened table's second return has head position eight (at transition 236). The original reaches the exact required successor frame at transition 486. The complete tapes and finite trace are retained in `search/evidence/scanner_concrete_counterexample.json`. This is a legal-frame simulation counterexample; it is not asserted to be a state on the unique blank run. The modified blank computation also fails to reach the intended clean seed in the tested million transitions, which is only a bounded observation.

Finally, all 194 HS controls occur in the union of the checked source runs, protected arithmetic blocks and exact startup. There is no unused control to prune in that proof domain. The known entry value $H = 1$ at the symmetric root does not remove the zero branch: source site 2 loops back after decrementing H, so the same guard is later entered with $H = 0$.

The negative experiments are scoped to their supplied table families, protected arithmetic interfaces, tape windows and step bounds. They are not a lower-bound proof. In particular, failure of a universal contract does not prove that every smaller machine with a different source or a more restricted reachable invariant is impossible. It does rule out using that failed certificate for the proposed unrestricted simulation.

7.1 Lessons from ZFC and ConciseIPRA

The ZFC sequence provides concrete techniques: use local entry-history bits to share decoder states, omit unnecessary zero pushes with a new proof-word compiler, and separate source semantics from literal startup. The no-push idea used here was already present in that work and in the PA196 research. The gain here is its verified combination with the cache-only controller at 194 states. State savings from different quotients cannot be added unless the final incoming contexts and all source outcomes still pass.

ConciseIPRA provides useful design principles for the logical front end, especially recovering broad rules from restricted primitive operations and making the certificate representation do some of the work. Its target, however, is iPRA and its size measures concern complete Python proof checkers. Those results do not establish a PA-consistency theorem or a two-symbol state budget for this machine. Replacing the enumerated calculus by iPRA, or reporting a smaller checker, would require an additional calibration before it could answer the present problem.

8 Reproduction, package contents and limitations

All construction and checking code uses Python 3.10 or later and its standard library. Run from the extracted package root with assertions enabled:

```
python3 verify.py
python3 verify.py --full
```

The default reconstructs both tables in a fresh temporary directory, checks all construction contracts and full startup, checks the hybrid command identities and exact graph novelty, and verifies recorded independent evidence against its bound machine and source. The full run also rederives the independent list lemmas and all literal-tape contracts. Neither command requires a network connection or the ProveIt checkout. `verification/release_check.json` records a fresh full replay of the delivered package. The corpus census may separately be rerun against the pinned repository using `comparison/audit_census.py`.

Directory	Contents
`machines/`	Dense/native tables, exact label maps, raw parents, source instructions and addresses, folds, both verification receipts.
`construction/`	Deterministic builder, frozen grammar, parent inputs, attributed runtime and source references, free-term command checks.
`verification/`	Independent literal-tape verifier, list-loop rules, replay and review records.
`comparison/`	Full 46-packet ledger, 28-class frontier, fresh census, exact prior tables up to 196 states and novelty checker.
`search/`	Portable bounded reduction experiments, machine/source inputs and negative results.
`references/logical/`	Attributed inherited ordinary calibration texts.
`report/`	This article, included LaTeX sources and the complete printed primary table.

To rebuild this PDF, run `pdflatex -interaction=nonstopmode -halt-on-error main.tex` twice from `report/`. The release manifest binds all delivered files except the manifest itself. Code, mathematical claims and negative-search scopes are stated separately throughout. The accomplished target is a materially different 194-state design. A valid 193-state machine remains open in this investigation.

References

[1] Vladimir Reshetnikov, *ProveIt*, snapshot `1e407ae841e4666f932777fad977ba784ebbcfd5`. https://github.com/VladimirReshetnikov/ProveIt/tree/1e407ae841e4666f932777fad977ba784ebbcfd5.

[2] *PA consistency machines*, the snapshot's `Computability/BusyBeaver/Research/PA/README.md`; research packet ledger and proof-status distinctions. See the full locators and hashes in `comparison/packet_ledger.json`.

[3] *A 194-state Turing machine calibrated to Con(PA)*, `PA/docs/pa194_conpa_2026-11-13/`; primary grammar, seven folds, separate-head calibration and independent physical runtime.

[4] *PA194C and PA194CS*, `PA/docs/pa194_new_designs_2026-11-13/`; cached controller and symmetric comparison. Multiple other supplied packets repeat PA194C.

[5] *PA194N*, `docs/incoming/pa194N_conpa_2026-11-13.zip`; no-push MP, total numerical stack identities and ten-entry proof-word compiler.

[6] James Chang, imported PA196/PA209 snapshots, `PA/jamesyc-upstream/`; numerical source, physical register engine, and separately stated internal and external logical endpoints.

[7] *Smaller machines for Con(ZFC)*, `Computability/BusyBeaver/Research/ZFC/README.md`; contextual folds, no-push transformations and exact-startup discipline.

[8] *Concise presentations of intuitionistic primitive recursive arithmetic*, `Logic/PrimitiveRecursiveArithmetic/ConciseIPRA/README.md`; native calculi and their proof-checker complexity measures.

A The complete PA194HS transition table

Each entry wDq' means write w, move in direction D, and enter q'. Label 194 is terminal. The dense table's state 0 is the blank start.

q	Read 0	Read 1	q	Read 0	Read 1	q	Read 0	Read 1	q	Read 0	Read 1
0	1R1	1R2	49	0L66	0L15	98	0R112	1R104	147	0R149	1R158
1	1R3	1R1	50	0R67	0L10	99	0R113	0L14	148	0R159	1R46
2	0R0	1R2	51	0R68	1R69	100	0R114	1L66	149	0R91	1R160
3	0R4	0R1	52	0R70	1R71	101	0R115	1R116	150	0R161	1R162
4	1R5	1L6	53	0R72	1R73	102	0R117	1R102	151	0R163	0L10
5	1L7	1L8	54	0R74	1R75	103	0R14	1R103	152	0R164	1R165
6	0L7	1L6	55	0L76	0R1	104	0L109	1L33	153	0R166	1R157
7	0L9	1L6	56	0R61	1R56	105	0R67	1R118	154	0R167	1R156
8	0L10	0L11	57	0L68	1L35	106	0R119	1L120	155	0R135	1R130
9	1R12	0L9	58	0R77	1R78	107	0R121	1R122	156	0R147	1R40
10	0L9	0L10	59	0R51	1R79	108	0R46	0L14	157	0R134	1R168
11	1L8	1L11	60	0L80	1R81	109	0R85	1R94	158	0R40	1L169
12	0R12	0R13	61	0R82	1R61	110	0R109	1L89	159	0L170	1R171
13	0L14	0R13	62	0L76	0R2	111	0R123	1R123	160	0R100	1R79
14	1L15	0L15	63	0R83	1R63	112	0L60	0R82	161	0R172	1R170
15	0L15	1L16	64	0R30	1R84	113	0R62	0R117	162	0R173	1R174
16	0L17	1L16	65	0R85	1R86	114	0R55	0R117	163	0R175	1R176
17	0L18	1L16	66	0R35	1R87	115	0L80	1R124	164	0R177	1R178
18	0L19	1L16	67	0R41	0L13	116	0R125	1R126	165	0L179	1L180
19	0L20	1L16	68	0R88	1R65	117	0R127	1R117	166	0R149	1R140
20	0L21	1L16	69	0R47	1R89	118	0R47	1R84	167	0R165	1R181
21	0L22	1L16	70	0R41	1R90	119	0R55	0R127	168	0L130	1R46
22	0L23	1L16	71	0R91	1R92	120	0R35	1R33	169	0R182	1R135
23	0R23	1R24	72	0R93	1L94	121	0R93	1R108	170	0R183	1R145
24	0R25	1R26	73	0R47	1R72	122	0R74	1R73	171	0R158	1R130
25	0R27	1R28	74	0R87	1R95	123	0R46	1R46	172	0R139	1R165
26	0R29	1R30	75	0R47	1R96	124	0R128	1R129	173	0R158	1R184
27	0R31	1R194	76	1R76	1L15	125	0R32	1R64	174	0R185	1R100
28	0L32	1R33	77	0R88	0L7	126	0R31	1R92	175	0R31	1R49
29	0R34	1R35	78	0R97	1R98	127	0R63	1R127	176	0R186	1R175
30	0L33	1L36	79	0R41	1R99	128	0R130	1R131	177	0R134	1R142
31	0R37	1R38	80	0R100	1R84	129	0R132	1R133	178	0R32	1R187
32	0R39	1R40	81	0R101	0L67	130	0R134	1R135	179	0R144	1R188
33	0R41	1R42	82	0R102	1R82	131	0R136	1R137	180	0R189	1R190
34	0R43	1R44	83	0R103	1R83	132	0R138	1R139	181	0R40	1R191
35	0R45	1R36	84	0R104	0L14	133	0R140	1R141	182	0R168	1R40
36	0R46	1R47	85	0R62	0R102	134	0R91	1R120	183	0R192	1R46
37	0R48	1R41	86	0R105	0L84	135	0R142	1R123	184	0R153	1R146
38	0R49	1R50	87	0R30	1R57	136	0R143	1R144	185	0R165	1R180
39	0R51	1R52	88	0L60	0R83	137	0R145	1R146	186	0R118	1R106
40	0R53	1R54	89	0R88	1L106	138	0R134	1R147	187	0R193	1R159
41	0R55	0R56	90	0R95	0L14	139	0R46	1R148	188	0R169	1R154
42	0R57	0L14	91	0R64	1R107	140	0R149	1L131	189	0R158	1R192
43	0R58	1R59	92	0R106	1R70	141	0R150	1R151	190	0R179	1R182
44	0R60	1R49	93	0L60	0R103	142	0R71	1R40	191	0R182	1R40
45	0R55	0R61	94	0L59	1R108	143	0R28	1R152	192	0R149	1R168
46	0R62	0R63	95	0L109	1R110	144	0L153	1L154	193	0R71	1R144
47	0L64	1L65	96	0R111	0L14	145	0L129	1R155			
48	0R45	1R50	97	0R112	1R89	146	0R156	1R157			

B All 46 research packets at the pinned snapshot

The entries record the packets' main contributions and stated outcomes; they do not relabel every supplied claim as a newly checked theorem. Full repository paths, README hashes and archive provenance are in the machine-readable ledger. "Incoming" entries are the five archives still in `docs/incoming`; other entries are filed under the PA research tree.

Packet	Contribution and stated outcome
1. `pa_unary_power_cover`	Unary power covering at PA strength (report of 27 October 2026). The U208 packets carry it as a baseline.
2. `U208_ConPA_Conditional_Context_2026-10-29`	U208 by a conditional tape-context reduction. States a 208-state candidate machine/u208.tm.
3. `U208_ConPA_Entry_Context_2026-10-29`	U208 by eliminating a state using entry context. States a 208-state machine machine/u208.tm.
4. `U208_ConPA_History_2026-10-29`	U208 by a finite-history decoder fold. States a 208-state candidate machine/u208.tm.
5. `U208_ConPA_2026-10-29`	U208 by a three-cell fold. States a 208-state candidate machine/u208.tm.
6. `u208_research_2026-10-29`	U208 research release. States two 208-state machines, machine/u208.tm and machine/variant_plain/u208.tm.
7. `Below_U208_Research_2026-10-29`	Research toward fewer than 208 states. States that no machine with 207 states or fewer was obtained.
8. `u208_comparison_research_2026-10-29`	Comparison of the 208-state proposals and a search below 208. States that the target was not achieved.
9. `ConPA_below208_research_2026-10-29`	Comparative audit and reduction search below 208. States that it does not establish a 207-state machine.
10. `pa_consistency_research_2026-10-29`	Audit and new source reductions. States that no validated machine below 208 states was obtained.
11. `pa208_research_2026-10-29`	Comparison and research below 208 states. States that it does not establish a valid machine below 208.
12. `pa_tm_research_2026-11-13`	Comparison and reduction research. States a new 196-state variant with shorter source paths and nothing below 196.
13. `pa_consistency_research_2026-11-13`	Comparison and diagonal-vacuity research. States a shorter axiom program with a 199-state implementation and nothing below 196.
14. `conpa_tm_comparison_and_reduction_search`	Comparison and reduction search. States two experimental 196-state command variants and nothing below 196.
15. `pa196_research_2026-11-13`	PA196 comparison, source factorization and proof obligations. States a source tree of 157 internal nodes and nothing below 196.
16. `pa_consistency_research`	Research and verification packet. States that no machine below 196 states has been validated.
17. `pa_tm_comparison_and_reductions`	Comparison and new rewrites, a research snapshot.
18. `pa_tm_extended_research_2026-11-13`	Comparison and refined reduction search. States that the smallest accepted count is 196.
19. `conpa_tm_comparison_and_search`	Comparison and a search below 196. States that none was obtained; files named 195 are rejected or diagnostic candidates.
20. `pa_consistency_reduction`	Comparison and reduction experiments. States two new 196-state variants and a shorter source program with a 203-state implementation.
21. `pa_consistency_comparison_2026_11_13`	Comparison and new reduction experiments. States that nothing below 196 was obtained.
22. `PA_consistency_TM_research_2026-11-13`	Comparison and new source reductions. States that no machine below 196 is claimed.
23. `pa_consistency_research_2026-11-13_2`	Comparison and search below 196. States two new layouts with 196 and 198 states and a command-level reduction.

Packet	Contribution and stated outcome
24. conpa_tm_study_2026-11-13	Comparison and reduction study. States that nothing below 196 was obtained.
25. pa_below196_research	Comparison and reduction experiments. States that nothing below 196 was obtained.
26. pa_consistency_comparison_and_new196_2026-11-13	Comparison and reduction research. States that it contains no accepted machine below 196.
27. pa_tm_research_2026-11-13_2	Comparison and reduction investigation. States same-size work, failed candidates and nothing below 196.
28. pa_consistency_cross_archive	Cross-archive constructions and reduction search. States that it does not certify a 195-state machine.
29. pa195_conpa_2026-11-13	PA195. States a 195-state machine pa195/pa195.tm and an article with a PA proof of the equivalence.
30. pa_consistency_reduction_audit_2026-11-14	Comparison and reduction audit. States nothing below 196 and no new theorem.
31. pa_consistency_research_2026-11-14	Comparison and NEG18. States an 18-instruction encoded negation with a compiled machine of 197 states.
32. PA195_ConPA_research_2026-11-14	A 195-state machine. States the primary machine_195S.tm and a second table machine_195C.tm.
33. pa195_followup_research	Comparison and searches below 195. States a graph-distinct 195-state machine M195S8 and nothing below 195.
34. pa194_conpa_2026-11-13	A 194-state machine. States the table construction/tables/pa194.tm and a proof of the equivalence in PA.
35. pa194_conpa_2026-11-13_2	PA194. States a 194-state machine pa194/pa194.tm and an article with a PA proof of the equivalence.
36. pa195_diagonal_combined_2026-11-13	PA195DC. States a different 195-state machine.
37. pa194C_conpa_2026-11-13_2	PA194C. States a materially different 194-state machine, an article with a PA proof of the equivalence, and nothing below 194.
38. pa194_new_designs_2026-11-13	Two new 194-state machines, PA194C and PA194CS. States that nothing below 194 was obtained.
39. pa194c_conpa	PA194C, a cached-head 194-state machine. States that no machine below 194 was established.
40. pa194c_conpa_2026-11-13_1	PA194C. States a different 194-state machine and an article with a PA proof of the equivalence.
41. pa194c_conpa_2026-11-13	PA194C, a cached-head 194-state machine. States an article with a PA proof of the equivalence and nothing below 194.
42. PA194R_research_2026-11-14 *Incoming*	Four 194-state tables R,F,A,X: final/entry known-zero decrements and a new decoder reassociation. X duplicates PA194CSZ in the Q packet. No verified sub-194 machine.
43. pa194N_conpa_2026-11-13 *Incoming*	194-state no-push modus-ponens controller. New ten-entry command compiler; original Shift. No verified sub-194 machine.
44. pa194_radix_family_2026-11-13 *Incoming*	194-state coefficients 5 and 6 via formula recoding, plus coefficient-4 reassociated M4R. No verified sub-194 machine.
45. pa194j_pa194d_conpa_2026-11-13 *Incoming*	Two principal 194-state machines: J with ten known-zero jumps, D with separate diagonal head Gamma(D) and 69 readers /9 folds. Additional less-verified siblings.
46. pa194q_conpa_2026-11-13 *Incoming*	194-state Q with exchanged curried quantifier-distribution premises; secondary CSZ combines symmetric Shift and known-zero jump. No verified sub-194 machine.

C All existing 194-state action-graph classes

Principal/companion means a machine presented as a result in a packet; research/supplementary and rejected have their weaker stated scopes. Multiple byte forms can represent the same graph. Neither PA194H nor PA194HS occurs in this prior list.

Class	Supplied role	Forms	Copies
PA194CSZ = PA194X	Principal/companion	2	10
PA194A	Principal/companion	2	4
PA194R	Principal/companion	2	4
PA194CS	Principal/companion	2	22
PA194N	Principal/companion	2	5
MPH.native research input	Research/supplement	1	1
PA194J	Principal/companion	2	2
M4R	Principal/companion	2	2
PA194_AD	Principal/companion	2	38
PA194CSZ_A5	Research/supplement	2	2
PA194-altC	Research/supplement	1	9
PA194Q	Principal/companion	3	3
REJECTED_node1_X	Rejected	1	1
PA194 (67 readers /7 folds)	Principal/companion	3	52
Rejected diagonal191 seven-zero scanner	Rejected	1	13
PA194CZ	Research/supplement	2	16
PA194CSZ_A3	Research/supplement	2	2
C_I_Z research input	Research/supplement	1	1
PA194D	Principal/companion	2	2
Gamma(D)_CSZA	Research/supplement	2	2
M6	Principal/companion	2	2
PA194CSZ_A6	Research/supplement	2	2
Rejected zerojump_103	Rejected	1	2
M5	Principal/companion	2	2
PA194 (68 readers /8 folds)	Principal/companion	2	25
CS_both_zero6 research input	Research/supplement	1	1
PA194C (repeated identical principal)	Principal/companion	2	31
PA194F	Principal/companion	2	5

Forms counts distinct complete file byte forms; *Copies* counts recognized occurrences.